

# Практическое занятие 29: Реализация регистрации и входа пользователя

## Цель занятия:

- Научиться реализовывать регистрацию новых пользователей в Django-приложении.
  - Настроить механизм аутентификации пользователей с помощью встроенной системы Django.
  - Понять, как создавать формы регистрации и входа, а также обрабатывать их данные.
  - Обеспечить возможность пользователям входить в систему и выходить из нее.
- 

## Задачи занятия

1. Введение в систему аутентификации Django
  2. Создание формы регистрации пользователя
  3. Реализация представления для регистрации
  4. Создание шаблона регистрации
  5. Настройка входа пользователя
  6. Создание шаблона входа
  7. Настройка выхода пользователя
  8. Добавление ссылок на регистрацию и вход в навигацию
  9. Тестирование функциональности
- 

## Задача 1: Введение в систему аутентификации Django

### 1.1. Что такое система аутентификации Django?

- **Django Authentication System** — встроенная система Django для управления пользователями, их регистрацией, входом и выходом.
- Предоставляет готовые модели, формы и представления для работы с пользователями.
- Позволяет быстро и безопасно внедрять аутентификацию в ваше приложение.

## 1.2. Почему мы используем встроенную систему аутентификации?

- **Безопасность:** Django тщательно протестировал и проверил свою систему аутентификации на безопасность.
  - **Удобство:** Многие задачи уже реализованы, вам нужно только настроить их под свои нужды.
  - **Расширяемость:** Систему можно расширять и настраивать под требования вашего приложения.
- 

## Задача 2: Создание формы регистрации пользователя

### 2.1. Использование встроенной формы `UserCreationForm`

Шаги:

1. Создайте файл `forms.py` в вашем приложении (если его нет).

```
touch events/forms.py
```

2. Импортируйте `UserCreationForm` и создайте свою форму на его основе:

```
# events/forms.py

from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User

class SignUpForm(UserCreationForm):
    email = forms.EmailField(max_length=254, required=True,
        help_text='Обязательное поле. Введите действительный адрес электронной почты.')

    class Meta:
        model = User
        fields = ('username', 'email', 'password1', 'password2')
```

Пояснение:

- **Наследуемся от `UserCreationForm`**, чтобы использовать встроенные методы и валидацию.

- Добавляем поле `email`, так как по умолчанию оно не включено.
  - Определяем класс `Meta`, чтобы указать, какие поля будут в форме.
- 

## Задача 3: Реализация представления для регистрации

### 3.1. Создание функции `signup` в `views.py`

Шаги:

1. Откройте `views.py` в вашем приложении `events`.
2. Импортируйте необходимые модули и формы:

```
# events/views.py

from django.shortcuts import render, redirect
from django.contrib.auth import login
from django.contrib import messages
from .forms import SignUpForm
```

3. Добавьте функцию `signup`:

```
# events/views.py

def signup(request):
    if request.method == 'POST':
        form = SignUpForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)
            messages.success(request, 'Регистрация прошла успешно!')
            return redirect('events:home') # Замените на вашу главную
            страницу
        else:
            messages.error(request, 'Пожалуйста, исправьте ошибки
            ниже.')
    else:
        form = SignUpForm()
    return render(request, 'events/signup.html', {'form': form})
```

Пояснение:

- Обработываем метод `POST`: Если форма отправлена, проверяем ее валидность.

- Сохраняем нового пользователя и выполняем автоматический вход с помощью `login(request, user)`.
  - Перенаправляем пользователя на главную страницу или другую страницу по вашему выбору.
  - При методе `GET` отображаем пустую форму регистрации.
- 

## Задача 4: Создание шаблона регистрации

### 4.1. Создание файла `signup.html`

Шаги:

1. Создайте файл `signup.html` в директории `templates/events/`:

```
mkdir -p events/templates/events
touch events/templates/events/signup.html
```

2. Добавьте следующий код в `signup.html`:

```
<!-- events/templates/events/signup.html -->

{% extends 'events/base.html' %}

{% block title %}Регистрация{% endblock %}

{% block content %}
    <h2>Регистрация</h2>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Зарегистрироваться</button>
    </form>
{% endblock %}
```

Пояснение:

- Используем базовый шаблон `base.html`.
  - Отображаем форму регистрации, используя `{{ form.as_p }}` для простого отображения.
-

# Задача 5: Настройка входа пользователя

## 5.1. Использование встроенных представлений Django

Django предоставляет готовые представления для входа и выхода пользователя.

**Шаги:**

1. Импортируйте необходимые модули в `urls.py`:

```
# events/urls.py

from django.urls import path
from django.contrib.auth import views as auth_views
from . import views

app_name = 'events'

urlpatterns = [
    # ... другие маршруты ...
    path('signup/', views.signup, name='signup'),
    path('login/',
        auth_views.LoginView.as_view(template_name='events/login.html'),
        name='login'),
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),
]
```

**Пояснение:**

- Используем `LoginView` и `LogoutView` из `django.contrib.auth.views`.
- Указываем `template_name` для `LoginView`, чтобы использовать наш шаблон.

## 5.2. Создание шаблона входа `login.html`

**Шаги:**

1. Создайте файл `login.html` в директории `templates/events/`:

```
touch events/templates/events/login.html
```

2. Добавьте следующий код в `login.html`:

```
<!-- events/templates/events/login.html -->

{% extends 'events/base.html' %}
```

```
{% block title %}Вход{% endblock %}

{% block content %}
    <h2>Вход</h2>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Войти</button>
    </form>
    <p>Нет аккаунта? <a href="{% url 'events:signup' %}">Зарегистрируйтесь</a></p>
{% endblock %}
```

#### Пояснение:

- Отображаем форму входа, предоставляем ссылку на регистрацию.

## Задача 6: Настройка выхода пользователя

### 6.1. Добавление ссылки на выход в навигацию

#### Шаги:

1. Откройте `base.html` и добавьте ссылки на вход и выход:

```
<!-- templates/base.html -->
```

```
{% if user.is_authenticated %} Hello, {{ user.username }}! {% csrf_token %} Logout {% else %} Login Sign Up {% endif %}
```

#### Пояснение:

- Проверяем, аутентифицирован ли пользователь, и отображаем соответствующие ссылки.
- Используем `user.is_authenticated`, который всегда доступен в шаблонах благодаря контекстному процессору.

## Задача 7: Настройка перенаправления после входа и выхода

## 7.1. Настройка URL перенаправления в `settings.py`

Шаги:

1. Откройте `settings.py` и добавьте следующие настройки:

```
# settings.py

LOGIN_REDIRECT_URL = 'events:home' # Замените на ваше имя маршрута для
главной страницы
LOGOUT_REDIRECT_URL = 'events:home' # Или другую страницу
```

Пояснение:

- `LOGIN_REDIRECT_URL` : URL, на который будет перенаправлен пользователь после успешного входа.
- `LOGOUT_REDIRECT_URL` : URL для перенаправления после выхода.

---

## Задача 8: Добавление ссылок на регистрацию и вход в навигацию

### 8.1. Обновление навигации в `base.html`

Мы уже сделали это в предыдущих шагах, но убедитесь, что ваша навигация выглядит примерно так:

```
<!-- templates/base.html -->

<nav>
  <!-- ... другие ссылки ... -->
  {% if user.is_authenticated %}
    Привет, {{ user.username }}!
    <a href="{% url 'events:logout' %}">Выйти</a>
  {% else %}
    <a href="{% url 'events:login' %}">Войти</a>
    <a href="{% url 'events:signup' %}">Регистрация</a>
  {% endif %}
</nav>
```

---

## Задача 9: Тестирование функциональности

## 9.1. Проверка регистрации

- Перейдите по адресу: <http://127.0.0.1:8000/signup/>
- Заполните форму регистрации, убедитесь, что происходит успешная регистрация и вход.

## 9.2. Проверка входа и выхода

- Выйдите из системы, перейдя по ссылке "Выйти".
- Перейдите по адресу: <http://127.0.0.1:8000/login/>
- Войдите с помощью созданных учетных данных.
- Убедитесь, что перенаправление происходит корректно.

---

## Дополнительные задания

### 1. Добавление валидации на уникальность email

**Цель:** Убедиться, что каждый email адрес регистрируется только один раз.

**Шаги:**

1. Создайте пользовательский валидатор для поля email в SignUpForm:

```
# events/forms.py

from django.core.exceptions import ValidationError

class SignUpForm(UserCreationForm):
    email = forms.EmailField(max_length=254, required=True,
        help_text='Обязательное поле. Введите действительный адрес электронной почты.')

    class Meta:
        model = User
        fields = ('username', 'email', 'password1', 'password2')

    def clean_email(self):
        email = self.cleaned_data.get('email')
        if User.objects.filter(email=email).exists():
            raise ValidationError('Пользователь с таким email уже существует.')
        return email
```

**Пояснение:**



- Проверяем, существует ли пользователь с таким же email.
- Вызываем `ValidationError`, если email уже используется.

## 2. Добавление возможности восстановления пароля

**Цель:** Позволить пользователям сбрасывать пароль, если они его забыли.

**Шаги:**

1. Добавьте маршруты для восстановления пароля в `urls.py`:

```
# events/urls.py

from django.contrib.auth import views as auth_views

urlpatterns += [
    path('password_reset/',
         auth_views.PasswordResetView.as_view(template_name='events/password_reset.html'), name='password_reset'),
    path('password_reset/done/',
         auth_views.PasswordResetDoneView.as_view(template_name='events/password_reset_done.html'), name='password_reset_done'),
    path('reset/<uidb64>/<token>/',
         auth_views.PasswordResetConfirmView.as_view(template_name='events/password_reset_confirm.html'), name='password_reset_confirm'),
    path('reset/done/',
         auth_views.PasswordResetCompleteView.as_view(template_name='events/password_reset_complete.html'), name='password_reset_complete'),
]
```

2. Создайте необходимые шаблоны и настройте отправку почты в `settings.py`.

**Пояснение:**

- В рамках данного занятия настройка восстановления пароля может быть сложной, так как требует настройки отправки электронной почты.
- Рекомендуется выполнить это задание после изучения соответствующих материалов.

---

## Заключение

**Поздравляем! Вы успешно:**

- Реализовали регистрацию новых пользователей в вашем приложении.

- Настроили систему аутентификации с использованием встроенных возможностей Django.
  - Создали формы и шаблоны для регистрации и входа пользователей.
  - Обеспечили возможность входа и выхода из системы.
- 

**Если у вас возникнут дополнительные вопросы или потребуется помощь с каким-либо шагом — обращайтесь!**

**Успехов в разработке!**

---

## **Вопросы для обсуждения**

- **Почему важно использовать встроенную систему аутентификации Django вместо создания собственной?**
  - Использование встроенной системы аутентификации Django обеспечивает высокую степень безопасности и надежности, так как она тщательно протестирована и поддерживается сообществом. Это экономит время разработки, позволяя избежать ошибок и уязвимостей, связанных с самостоятельной реализацией механизмов аутентификации. Кроме того, встроенная система легко интегрируется с другими компонентами Django, обеспечивая совместимость и расширяемость.
  - **Как обеспечить безопасность данных при регистрации и входе пользователей?**
  - Для обеспечения безопасности данных при регистрации и входе пользователей следует использовать HTTPS для шифрования передаваемых данных, применять защиту от CSRF-атак с помощью токенов, использовать сильные хеши для паролей (Django автоматически хеширует пароли пользователей), а также внедрять ограничения на количество попыток входа для предотвращения атак перебором. Дополнительно рекомендуется регулярно обновлять зависимости и следить за уязвимостями в используемых пакетах.
  - **Какие преимущества предоставляет использование классов-представлений (Class-Based Views) для реализации регистрации и входа пользователей?**
  - Классы-представления в Django предлагают более структурированный и повторно используемый подход к разработке, позволяя легко наследоваться и расширять функциональность. Они облегчают организацию кода, делают его более читаемым и поддерживаемым. Использование классов-представлений также позволяет эффективно использовать миксины и встроенные методы, что ускоряет процесс разработки и снижает вероятность ошибок.
-

## Полезные ресурсы

- [Документация Django: Аутентификация](#)
- [Документация Django: Форма регистрации пользователей](#)
- [Видео-тutorial по регистрации и входу в Django](#)
- [Django Girls Tutorial: Регистрация пользователей](#)